Floor Plan Implementation Guide

Replicating the StayHere web client's floor-plan viewer on the Android app — data contract, geometry, and step-by-step rendering (Kotlin)

Audience: Android / Kotlin mobile engineering team
Source of truth: StayHere monorepo — PropertyService (API) & StayHereClientApp (reference web implementation)
Version 1.0 — September 2026

ACCURACY NOTE

This guide documents the **actual** JSON contract returned by the StayHere API and the **actual** rendering logic used by the web client (`StayHereClientApp/src/components/floorplan/`). Every field name, type, and coordinate convention below is taken directly from the backend C# source and the React/SVG implementation — nothing here is guessed. The one exception is the sample JSON in Section 3.6, which is clearly labelled as an illustrative example (no seeded floor-plan fixture exists in the repo to quote verbatim).

Contents

1. Overview & Goal
2. Tools & Libraries You'll Need
3. The Data Contract — What the API Returns
4. Furniture Catalog Contract
5. Kotlin Data Models
6. Networking Setup (Retrofit)
7. Step-by-Step — Rendering the 2D Floor Plan
8. Step-by-Step — Adding Furniture (2.5D overlay)
9. Going Further — True 3D Parity (optional)
10. Testing Your Implementation
11. Appendix A — Full Field Reference
12. Appendix B — Full Furniture Catalog
13. Appendix C — Quick-Reference Cheat Sheet

1. Overview & Goal

StayHere listings can have a **digitized floor plan** attached — an owner uploads a floor plan image, traces room outlines and door/window openings over it in the owner portal, and the result is stored on the listing as real-world geometry (in meters), not as a picture. The web client (`StayHereClientApp`) then renders that geometry three ways: a flat top-down **2D** SVG view, a static-camera **3D image** render, and an interactive **3D Live** scene where a visitor can walk around and even rearrange furniture.

This guide's goal is to get the Android app to the same starting point: fetch a listing, read its `floorPlan` field, and render a faithful **2D top-down floor plan** — matching the web app's "2D" tab exactly (room outlines, walls, doors, windows, room labels) — plus an optional furniture overlay. Section 9 discusses what true 3D parity would additionally require, so you can scope that as a follow-up phase rather than blocking the MVP on it.

> **KEY IDEA**
>
> The floor plan is **not an image you draw on the screen**. It is structured geometry (room polygons, wall openings) in meters that *you* draw — using whatever native drawing API you choose (Canvas / Compose Canvas). The original uploaded floor-plan photo is not needed for rendering and typically isn't even returned by this endpoint.

2. Tools & Libraries You'll Need

All of the following are standard, actively-maintained Android libraries — no exotic dependencies required for the 2D + furniture-overlay MVP described in this guide.

Purpose	Recommended library	Notes
UI framework	Jetpack Compose	Compose's Canvas composable is the most direct match for the web app's SVG drawing model used here.
Networking	Retrofit + OkHttp	Same pattern the rest of the codebase would use for any REST client.
JSON parsing	kotlinx.serialization (or Moshi)	Either works; this guide uses kotlinx.serialization since the API has simple, flat DTOs.
Coroutines	kotlinx.coroutines	For suspend-based API calls from Retrofit.
Color parsing	android.graphics.Color.parseColor()	Furniture catalog colors are plain hex strings, e.g. "#F5A623".
(Optional, Section 9)	Filament or SceneView	Only needed if you later pursue true 3D parity with the web app's react-three-fiber scene.

Minimum Kotlin/Android versions: nothing here requires anything beyond a modern Compose setup (Kotlin 1.9+, Compose BOM 2024.x+, minSdk 24+).

3. The Data Contract — What the API Returns

A listing's floor plan travels inside the normal single-listing response: `GET /property/listings/{id}`. The response is one JSON object whose `floorPlan` field is either a full floor-plan object, or literally `null` if the owner never uploaded one.

3.1 Where it lives in the listing response

```
GET /property/listings/{id}

{
  "id": "...",
  "listingCode": "...",
  "title": "...",
  ...                       // all the normal listing fields
  "floorPlan": { ... }      // <-- see 3.2, or null if none uploaded (see 3.5)
}
```

3.2 FloorPlanDto — the top-level floor-plan object

Field	Type	Description
`rooms`	`FloorPlanRoomDto[]`	Every room's outline polygon. See 3.3.
`openings`	`FloorPlanOpeningDto[]`	Every door/window cut into a wall. See 3.3.
`metersPerPixel`	`number (double)`	Audit/provenance value only — the scale used at digitization time to convert the source image into meters. You do NOT need this to render; room/opening coordinates are already in meters.
`sourceFileKey`	`string`	Storage key of the originally uploaded floor-plan image (not needed for rendering).
`sourceFileType`	`string`	MIME type of that source file, e.g. "image/png".
`digitizedAt`	`string (ISO-8601 datetime)`	When the owner/admin traced this floor plan.
`digitizedByRole`	`string`	"owner" or "admin".
`aiProvider`	`string? (nullable)`	Which AI provider assisted digitization, if any (e.g. "gemini"). Informational only.
`hotspots`	`FloorPlanHotspotDto[]? (nullable)`	Named room-to-room navigation waypoints for the 3D scene. Null/empty on plans digitized before 3D support existed — see 8.2 for the fallback.
`allowedCatalogIds`	`string[]? (nullable)`	Which furniture-catalog item ids are appropriate for this floor plan. Null/empty means "no restriction / not set".
`defaultLayout`	`FloorPlanLayoutItemDto[]? (nullable)`	The furniture placement to show by default. Null/empty on older plans — see 8.2 for the auto-seed fallback.

> **NOTE**
>
> **Everything you need for the 2D view (Section 7) is just `rooms` and `openings`.** The hotspot/catalog/layout fields only matter once you add the furniture overlay (Section 8) or go further into 3D (Section 9).

3.3 Room, opening, hotspot and layout-item shapes

FloorPlanRoomDto

Field	Type	Description
`id`	`string`	Stable room id, referenced by openings' RoomId analogues and by layout items.
`label`	`string`	Human room name, e.g. "Bedroom", "Kitchen".
`points`	`number[][]`	The room's outline as an ordered polygon: each element is a 2-item array `[x, z]`, in **meters**, on the floor plane. Closed implicitly — the last point connects back to the first.

FloorPlanOpeningDto

Field	Type	Description
`id`	`string`	Opening id.
`from`	`number[2]`	Wall-segment start point, [x, z] meters.
`to`	`number[2]`	Wall-segment end point, [x, z] meters.
`position`	`number (0.0–1.0)`	Fraction of the distance from "from" to "to" where this opening is centered — NOT a screen coordinate.
`widthMeters`	`number`	The opening's width, in meters, centered on "position".
`type`	`string`	"door" or "window".

> **NOTE**
>
> Openings are identified by their **wall endpoints** (from/to), not by "room + edge index". This is deliberate: two adjoining rooms share one physical wall, and this scheme lets a single opening record apply to that shared wall regardless of which room's polygon you're currently drawing. Section 7 Step 4 shows how to match openings to wall segments.

FloorPlanHotspotDto (3D navigation only — not needed for the 2D view)

Field	Type	Description
`id`	`string`	Hotspot id.
`roomId`	`string`	Which room this waypoint belongs to.
`label`	`string`	Display label, e.g. "Bedroom" (used for a "fly to room" list in 3D).
`order`	`int`	Display/navigation order.

FloorPlanLayoutItemDto (used by the furniture overlay, Section 8)

Field	Type	Description
`catalogId`	`string`	References an id in the furniture catalog — see Section 4.
`roomId`	`string`	Which room this piece of furniture is placed in.
`position`	`number[3]`	[x, y, z] meters. x/z are the floor-plane position; y is height off the floor (almost always 0 for floor-standing furniture).
`rotationY`	`number (degrees)`	Rotation around the vertical axis, in degrees.

3.4 The coordinate system — read this carefully

This is the single most important fact in this whole guide, so it's worth stating plainly: **every coordinate in `rooms` and `openings` is a real-world distance in meters, on an X/Z floor plane — not pixels, and not a 0–1 normalized fraction.** The backend's domain comment states this explicitly:

```
"Rooms/openings are stored in real-world meters, already converted from the
AI's fraction-of-image coordinates using MetersPerPixel at digitization time,
so no client needs the source image dimensions to render this geometry."
    — src/Domain/Entities/PropertyValueTypes.cs
```

- A point `[3.5, 2.0]` means 3.5 meters along X and 2.0 meters along Z from some arbitrary origin the digitizer chose — it is **not** tied to your screen size.
- "Y" (the third value, only present on furniture `position`, Section 3.3) is height off the floor — it does not appear in room/opening 2D points at all.
- There is no separate "image width/height" you need to fetch to make sense of these numbers. `metersPerPixel` was only relevant at digitization time (converting the uploaded image's pixels into these meter coordinates) — you can ignore it entirely for rendering.
- Because it's real-world meters, **you choose your own on-screen scale** (e.g. how many dp represent one meter) to fit the device screen. The web app hardcodes `PX_PER_METER = 40` CSS pixels per meter for its SVG view — see Section 7 Step 2 for the Android equivalent.

3.5 The "no floor plan uploaded" case

If a listing has no digitized floor plan, the response still includes the key — its value is JSON `null`, not an omitted field and not an empty object:

```
{
  "id": "...",
  ...
  "floorPlan": null
}
```

In Kotlin this maps naturally to a nullable field (Section 5) — always check for null before trying to render, and show an empty/placeholder state instead (Section 7 Step 6).

3.6 Example payload (illustrative)

> **ILLUSTRATIVE — NOT FROM THE CODEBASE**
>
> No seeded floor-plan fixture or test payload exists anywhere in the repository to quote verbatim. The JSON below was constructed by hand from the confirmed DTO shapes above, purely to give you something concrete to test against (Section 10). Treat it as a worked example, not as data pulled from a real listing.

```
{
  "rooms": [
    { "id": "room-bedroom", "label": "Bedroom",
      "points": [[0,0],[3.5,0],[3.5,4.0],[0,4.0]] },
    { "id": "room-bath", "label": "Bathroom",
      "points": [[3.5,0],[5.0,0],[5.0,2.2],[3.5,2.2]] }
  ],
  "openings": [
    { "id": "op-door-1", "from": [3.5,0.8], "to": [3.5,1.6],
      "position": 0.5, "widthMeters": 0.8, "type": "door" },
    { "id": "op-window-1", "from": [0,0], "to": [3.5,0],
      "position": 0.3, "widthMeters": 1.2, "type": "window" }
  ],
  "metersPerPixel": 0.0125,
  "sourceFileKey": "floorplans/{listingId}/original.png",
  "sourceFileType": "image/png",
  "digitizedAt": "2026-08-15T09:30:00Z",
  "digitizedByRole": "owner",
  "aiProvider": "gemini",
  "hotspots": [
    { "id": "hs-bedroom", "roomId": "room-bedroom", "label": "Bedroom", "order": 1 },
    { "id": "hs-bath", "roomId": "room-bath", "label": "Bathroom", "order": 2 }
  ],
  "allowedCatalogIds": ["bed_double","wardrobe","toilet","bathroom_sink","shower"],
  "defaultLayout": [
    { "catalogId": "bed_double", "roomId": "room-bedroom",
      "position": [1.75, 0, 2.0], "rotationY": 0 },
    { "catalogId": "toilet", "roomId": "room-bath",
      "position": [4.25, 0, 1.1], "rotationY": 90 }
  ]
}
```

4. Furniture Catalog Contract

Placed furniture (Section 3.3, `defaultLayout`) only tells you *which catalog item*, *where*, and *rotated how much*. The actual size/color/icon of each piece of furniture comes from a separate, static reference endpoint — the furniture catalog. It rarely changes and is safe to cache for the app's lifetime.

4.1 Endpoint

```
GET /static/furniture-catalog

-> 200 OK
-> Content-Type: application/json
-> Body: a FLAT array (no pagination, no envelope) of every catalog item
```

No query parameters, no auth required, no pagination — it always returns the full list (15 items as of this writing, see Appendix B).

4.2 FurnitureCatalogItemDto shape

Field	Type	Description
`id`	`string`	Stable id, e.g. "bed_double" — this is what defaultLayout.catalogId references.
`name`	`string`	Human display name, e.g. "Double Bed".
`category`	`string`	One of: bedroom, living, dining, kitchen, bathroom.
`footprint`	`number[2]`	**[width, depth]** in meters, on the floor plane (NOT [width, height]).
`height`	`number`	Height in meters — separate from footprint, used for the vertical extent of the 3D mesh.
`color`	`string (hex)`	e.g. "#F5A623" — a fill color for the item's shape (procedural mesh material color, but works equally well as a flat-fill color in a 2D overlay).
`icon`	`string (emoji)`	e.g. a bed emoji character for "bed_double" — handy as a quick visual label if you don't want to draw exact furniture silhouettes.

> **NOTE**
>
> There is **no** "placed furniture" endpoint or DTO. Placement always comes from a floor plan's own `defaultLayout` (Section 3.3) — the catalog only supplies the shared, reusable definition of each furniture *type*.

5. Kotlin Data Models

These mirror the JSON contract in Section 3 & 4 field-for-field, using `kotlinx.serialization`. Field names use Kotlin camelCase, which already matches the JSON keys exactly (no `@SerialName` mapping needed) because the backend serializes with the same convention.

```
import kotlinx.serialization.Serializable

@Serializable
data class FloorPlanRoomDto(
    val id: String,
    val label: String,
    val points: List<List<Double>>   // each inner list = [x, z] meters
)

@Serializable
data class FloorPlanOpeningDto(
    val id: String,
    val from: List<Double>,          // [x, z] meters
    val to: List<Double>,            // [x, z] meters
    val position: Double,            // 0.0-1.0 fraction from `from` to `to`
    val widthMeters: Double,
    val type: String                 // "door" | "window"
)

@Serializable
data class FloorPlanHotspotDto(
    val id: String,
    val roomId: String,
    val label: String,
    val order: Int
)

@Serializable
data class FloorPlanLayoutItemDto(
    val catalogId: String,
    val roomId: String,
    val position: List<Double>,      // [x, y, z] meters
    val rotationY: Double            // degrees
)
```

```
@Serializable
data class FloorPlanDto(
    val rooms: List<FloorPlanRoomDto>,
    val openings: List<FloorPlanOpeningDto>,
    val metersPerPixel: Double,
    val sourceFileKey: String,
    val sourceFileType: String,
    val digitizedAt: String,
    val digitizedByRole: String,
    val aiProvider: String? = null,
    val hotspots: List<FloorPlanHotspotDto>? = null,
    val allowedCatalogIds: List<String>? = null,
    val defaultLayout: List<FloorPlanLayoutItemDto>? = null
)

@Serializable
data class FurnitureCatalogItemDto(
    val id: String,
    val name: String,
    val category: String,
    val footprint: List<Double>,     // [width, depth] meters
    val height: Double,              // meters
    val color: String,               // hex, e.g. "#F5A623"
    val icon: String                 // emoji, e.g. "\uD83D\uDECF\uFE0F"
)

// Trimmed to the fields this guide needs; add the rest of the listing's
// fields as your app requires them.
@Serializable
data class ListingDto(
    val id: String,
    val title: String,
    val floorPlan: FloorPlanDto? = null
)
```

NOTE

`points`/`from`/`to` are `List<Double>` of length 2 (or 3 for furniture position), not a typed `Point` object — that matches the backend's own "double[]" representation. It's convenient to add tiny extension helpers, e.g. `fun List<Double>.x() = this[0]` / `.z() = this[1]`, to keep call sites readable.

6. Networking Setup (Retrofit)

A small Retrofit interface covering both endpoints this guide uses:

```
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Query

interface StayHereApi {

    @GET("listings/{id}")
    suspend fun getListing(@Path("id") id: String): ListingDto

    @GET("furniture-catalog")
    suspend fun getFurnitureCatalog(): List<FurnitureCatalogItemDto>
}
```

`listings/{id}` is served by **PropertyService**; `furniture-catalog` is served by **StaticDataService** — two different backend services in this system, so in practice you'll want two Retrofit instances (one per base URL), or a single instance pointed at a gateway that fronts both.

```
import kotlinx.serialization.json.Json
import okhttp3.MediaType.Companion.toMediaType
import retrofit2.Retrofit
import retrofit2.converter.kotlinx.serialization.asConverterFactory

val json = Json { ignoreUnknownKeys = true }

fun buildApi(baseUrl: String): StayHereApi =
    Retrofit.Builder()
        .baseUrl(baseUrl)   // must end with a trailing slash, e.g. ".../api/"
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()
        .create(StayHereApi::class.java)

// Two clients, one per backend service:
val propertyApi = buildApi(BuildConfig.PROPERTY_API_BASE)
val staticApi   = buildApi(BuildConfig.STATIC_API_BASE)
```

> **NOTE**
>
> **Testing against a local backend from the Android emulator:** the emulator can't resolve your host machine's `localhost` — use `10.0.2.2` instead. E.g. if `PropertyService` is running locally on port 7101, use `http://10.0.2.2:7101/api/` as PROPERTY_API_BASE. On a physical device on the same Wi-Fi network, use your machine's LAN IP instead.

Fetching a listing and its floor plan, end to end:

```
val listing = propertyApi.getListing(listingId)
val floorPlan = listing.floorPlan   // null if none uploaded — see Section 7 Step 6

if (floorPlan != null) {
    val catalog = staticApi.getFurnitureCatalog()   // only needed if showing furniture, Section 8
    // ... render (Section 7 and/or 8)
}
```

7. Step-by-Step — Rendering the 2D Floor Plan

This section walks through replicating the web app's "2D" tab (`StayHereClientApp/src/components/floorplan/FloorPlan2D.tsx`) using Jetpack Compose's `Canvas`. The web app draws this as flat SVG with no per-room-type coloring, no area labels, and no zoom/pan — matching that exactly is actually less work than you might expect, and gives you pixel-for-pixel parity with what customers already see on the website.

STEP 1 Compute the scene bounds

Every room's points are in an arbitrary meter coordinate space — before drawing anything, find the bounding box across **all** rooms so you know how large a canvas you need and how to offset every point so the drawing starts near (0,0).

```
data class Bounds(val minX: Double, val minZ: Double, val maxX: Double, val maxZ: Double) {
    val width get() = maxX - minX
    val depth get() = maxZ - minZ
}

fun computeBounds(rooms: List<FloorPlanRoomDto>): Bounds {
    val xs = rooms.flatMap { r -> r.points.map { it[0] } }
    val zs = rooms.flatMap { r -> r.points.map { it[1] } }
    return Bounds(xs.min(), zs.min(), xs.max(), zs.max())
}
```

STEP 2 Choose a scale (dp per meter) and build the coordinate transform

The web app hardcodes `PX_PER_METER = 40` CSS pixels per meter (`StayHereClientApp/src/lib/floorPlanGeometry.ts`). On Android, pick a similar constant in dp, plus a small padding, and convert every meter coordinate to a Compose `Offset` the same way:

```
const val PX_PER_METER = 120f   // tune this; higher = more zoomed in
const val PADDING_PX = 30f

fun toOffset(x: Double, z: Double, bounds: Bounds): Offset = Offset(
    ((x - bounds.minX) * PX_PER_METER + PADDING_PX).toFloat(),
    ((z - bounds.minZ) * PX_PER_METER + PADDING_PX).toFloat()
)

// Canvas size to request, so the whole floor plan fits without cropping:
fun canvasSize(bounds: Bounds): Size = Size(
    (bounds.width * PX_PER_METER + PADDING_PX * 2).toFloat(),
    (bounds.depth * PX_PER_METER + PADDING_PX * 2).toFloat()
)
```

> **NOTE**
>
> Web "x, z" → Compose "x, y": the web app calls the second axis "z" because it's really a 3D scene's floor plane viewed from above. In 2D Compose there's no third dimension, so it's perfectly fine (and clearer) to just treat that second value as your Compose y — the math above already does this for you.

STEP 3 Draw each room's polygon

The web app fills every room with a single flat color and no border — `fill="#F8FAFC" stroke="none"` (`FloorPlan2D.tsx`). There is **no per-room-type color coding** (bedroom vs. kitchen look the same) in the current web implementation, so match that unless product asks you to diverge from it.

```
val ROOM_FILL = Color(0xFFF8FAFC)

fun DrawScope.drawRoom(room: FloorPlanRoomDto, bounds: Bounds) {
    val path = Path().apply {
        room.points.forEachIndexed { i, p ->
            val o = toOffset(p[0], p[1], bounds)
            if (i == 0) moveTo(o.x, o.y) else lineTo(o.x, o.y)
        }
        close()
    }
    drawPath(path, color = ROOM_FILL)
}
```

STEP 4 Build wall segments and draw doors/windows

This is the trickiest part, and it's worth understanding *why* the web app does it this way: two adjoining rooms each list the shared wall as part of their own polygon, so drawing every room's raw edges naively would draw every shared wall twice. The web app (`StayHereClientApp/src/lib/roomGeometry.ts`) first collapses shared edges into one wall list, then cuts each opening into whichever wall segment its endpoints match.

4a — Collect every room's edges, deduplicating shared walls

```
data class WallSegment(val from: List<Double>, val to: List<Double>)

// Round + order-independent key so a wall shared by two rooms collapses to one entry.
fun wallKey(a: List<Double>, b: List<Double>): String {
    fun r(v: Double) = kotlin.math.round(v * 100) / 100.0   // ~1cm precision
    val p1 = "${'$'}{r(a[0])},${'$'}{r(a[1])}"
    val p2 = "${'$'}{r(b[0])},${'$'}{r(b[1])}"
    return if (p1 < p2) "$p1|$p2" else "$p2|$p1"
}

fun collectWallSegments(rooms: List<FloorPlanRoomDto>): List<WallSegment> {
    val seen = LinkedHashMap<String, WallSegment>()
    for (room in rooms) {
        val pts = room.points
        for (i in pts.indices) {
            val a = pts[i]
            val b = pts[(i + 1) % pts.size]   // wrap last -> first, closing the polygon
            seen.putIfAbsent(wallKey(a, b), WallSegment(a, b))
        }
    }
    return seen.values.toList()
}
```

4b — Split each wall into solid / door / window pieces

For each opening, find the wall segment whose endpoints match its `from`/`to` (openings are defined by wall endpoints for exactly this reason — see Section 3.3), then compute where along that segment the opening sits using `position` and `widthMeters`.

```
sealed class WallPiece {
    data class Solid(val from: List<Double>, val to: List<Double>) : WallPiece()
    data class DoorOrWindow(
        val from: List<Double>, val to: List<Double>, val type: String
    ) : WallPiece()
}

fun buildWallPieces(
    segments: List<WallSegment>,
    openings: List<FloorPlanOpeningDto>
): List<WallPiece> {
    val pieces = mutableListOf<WallPiece>()
    for (seg in segments) {
        val onThisWall = openings.filter {
            wallKey(it.from, it.to) == wallKey(seg.from, seg.to)
        }
        if (onThisWall.isEmpty()) {
            pieces += WallPiece.Solid(seg.from, seg.to); continue
        }
        // For a wall with N openings: walk the wall length, emitting a Solid piece
        // before/between/after each opening, and a Door/Window piece for the opening itself.
        val length = distance(seg.from, seg.to)
        var cursor = 0.0
        for (op in onThisWall.sortedBy { it.position }) {
            val center = op.position * length
            val halfW = op.widthMeters / 2.0
            val startFrac = ((center - halfW).coerceAtLeast(cursor)) / length
            val endFrac = ((center + halfW).coerceAtMost(length)) / length
            if (startFrac > cursor / length) {
                pieces += WallPiece.Solid(lerp(seg.from, seg.to, cursor / length), lerp(seg.from, seg.to, startFrac))
            }
            pieces += WallPiece.DoorOrWindow(
                lerp(seg.from, seg.to, startFrac), lerp(seg.from, seg.to, endFrac), op.type
            )
            cursor = endFrac * length
        }
        if (cursor < length) {
            pieces += WallPiece.Solid(lerp(seg.from, seg.to, cursor / length), seg.to)
        }
    }
    return pieces
}


fun distance(a: List<Double>, b: List<Double>) =
    kotlin.math.hypot(b[0] - a[0], b[1] - a[1])

fun lerp(a: List<Double>, b: List<Double>, t: Double) =
    listOf(a[0] + (b[0] - a[0]) * t, a[1] + (b[1] - a[1]) * t)
```

4c — Draw each piece with the matching web-app style

Piece type	Web app style (source of truth)	Meaning
`Solid wall`	`Single black line, stroke #0F172A, width 5px`	A plain wall segment.
`Window`	`Three parallel blue lines, stroke #3B82F6`	Center line + two offset lines — a window mullion symbol.
`Door`	`Short brown leaf line #8A6D4E, plus a dashed arc (strokeDasharray "3 2")`	The door leaf, plus the classic architectural door-swing arc.

```
val WALL_COLOR = Color(0xFF0F172A)
val WINDOW_COLOR = Color(0xFF3B82F6)
val DOOR_COLOR = Color(0xFF8A6D4E)

fun DrawScope.drawWallPiece(piece: WallPiece, bounds: Bounds) {
    when (piece) {
        is WallPiece.Solid -> {
            val a = toOffset(piece.from[0], piece.from[1], bounds)
            val b = toOffset(piece.to[0], piece.to[1], bounds)
            drawLine(WALL_COLOR, a, b, strokeWidth = 5f)
        }
        is WallPiece.DoorOrWindow -> {
            val a = toOffset(piece.from[0], piece.from[1], bounds)
            val b = toOffset(piece.to[0], piece.to[1], bounds)
            if (piece.type == "window") {
                // Center line + two parallel offset lines = window symbol.
                drawLine(WINDOW_COLOR, a, b, strokeWidth = 2f)
                val nx = -(b.y - a.y) * 0.06f
                val ny = (b.x - a.x) * 0.06f
                drawLine(WINDOW_COLOR, Offset(a.x + nx, a.y + ny), Offset(b.x + nx, b.y + ny), strokeWidth = 1.4f)
                drawLine(WINDOW_COLOR, Offset(a.x - nx, a.y - ny), Offset(b.x - nx, b.y - ny), strokeWidth = 1.4f)
            } else { // "door"
                drawLine(DOOR_COLOR, a, b, strokeWidth = 3f)
                // Swing arc: a quarter-circle from the door leaf, radius = door width.
                val radius = kotlin.math.hypot((b.x - a.x).toDouble(), (b.y - a.y).toDouble()).toFloat()
                drawArc(
                    color = DOOR_COLOR.copy(alpha = 0.6f),
                    startAngle = 0f, sweepAngle = 90f, useCenter = false,
                    topLeft = Offset(a.x - radius, a.y - radius),
                    size = Size(radius * 2, radius * 2),
                    style = Stroke(width = 1f, pathEffect = PathEffect.dashPathEffect(floatArrayOf(6f, 4f)))
                )
            }
        }
    }
}
```

NOTE

The exact door-swing arc math (which corner it pivots from, which quadrant it sweeps) depends on wall orientation — the snippet above is a starting point, not a pixel-exact port. If exact swing-arc geometry matters for your first release, budget time to tune the angle per wall orientation, or simplify to just drawing the door-leaf line without the arc for v1.

STEP 5 Draw room labels

The web app draws each room's label at its polygon's centroid, uppercased, bold, dark navy text (`fontSize=13 fontWeight=600 fill="#1E293B"`). No per-room area (sqm) label is shown — a `polygonArea()` helper exists in the web codebase but is not currently wired into any visible UI, so there is nothing to match there.

```
fun polygonCentroid(points: List<List<Double>>): List<Double> {
    var cx = 0.0; var cz = 0.0
    points.forEach { cx += it[0]; cz += it[1] }
    return listOf(cx / points.size, cz / points.size)
}

fun DrawScope.drawRoomLabel(room: FloorPlanRoomDto, bounds: Bounds, textMeasurer: TextMeasurer) {
    val centroid = polygonCentroid(room.points)
    val o = toOffset(centroid[0], centroid[1], bounds)
    val text = textMeasurer.measure(
        room.label.uppercase(),
        style = TextStyle(fontSize = 13.sp, fontWeight = FontWeight.SemiBold, color = Color(0xFF1E293B))
    )
    drawText(text, topLeft = Offset(o.x - text.size.width / 2f, o.y - text.size.height / 2f))
}
```

STEP 6 Handle the "no floor plan" state

Always check `listing.floorPlan` for null before attempting any of the above (Section 3.5) — show a simple empty state instead:

```
@Composable
fun FloorPlanOrEmptyState(floorPlan: FloorPlanDto?) {
    if (floorPlan == null) {
        Text("No floor plan available for this listing.", style = MaterialTheme.typography.bodyMedium)
    } else {
        FloorPlan2DView(floorPlan)
    }
}
```

8. Step-by-Step — Adding Furniture (2.5D Overlay)

On the web app, furniture is only drawn in the "3D Live" and "3D Image" tabs — the flat 2D SVG tab shows rooms/walls/openings only, no furniture. Full 3D parity is a much bigger lift (Section 9). This section instead shows a pragmatic **2.5D** approach: draw each placed furniture item as a simple, correctly-sized-and-rotated top-down rectangle (using its real `footprint`) directly on top of the same 2D floor plan from Section 7. This communicates the same information — what furniture is where — without needing a 3D engine.

STEP 1 Fetch and cache the furniture catalog

The web app fetches this once per session and caches it in memory (`StayHereClientApp/src/lib/furnitureCatalogClient.ts`) since it rarely changes. Do the same — an in-memory cache backed by a simple singleton or a `StateFlow` is enough; there's no need to persist it to disk.

```
object FurnitureCatalogCache {
    private var cache: List<FurnitureCatalogItemDto>? = null

    suspend fun get(api: StayHereApi): List<FurnitureCatalogItemDto> {
        cache?.let { return it }
        val fetched = api.getFurnitureCatalog()
        cache = fetched
        return fetched
    }
}
```

STEP 2 Resolve the layout to draw

Use `floorPlan.defaultLayout` when present. If it's null/empty (an older, pre-3D-support floor plan), the web app auto-generates starter furniture per room by keyword-matching the room label — e.g. a label matching `/bed/i` gets a bed + wardrobe, `/kitchen/i` gets a counter + fridge + stove (`StayHereClientApp/src/lib/autoSeed.ts`). Replicate a simplified version of that fallback so old listings aren't left looking empty:

```
fun resolveLayout(floorPlan: FloorPlanDto): List<FloorPlanLayoutItemDto> {
    if (!floorPlan.defaultLayout.isNullOrEmpty()) return floorPlan.defaultLayout

    // Fallback for older floor plans with no saved layout: seed one item per
    // keyword match, centered in that room's bounding box.
    val seeds = mapOf(
        Regex("bed", RegexOption.IGNORE_CASE) to listOf("bed_double" to 0.0),
        Regex("kitchen", RegexOption.IGNORE_CASE) to listOf("kitchen_counter" to 0.0, "fridge" to 90.0),
        Regex("bath", RegexOption.IGNORE_CASE) to listOf("toilet" to 0.0, "shower" to 90.0),
        Regex("living", RegexOption.IGNORE_CASE) to listOf("sofa" to 0.0, "coffee_table" to 0.0),
        Regex("dining", RegexOption.IGNORE_CASE) to listOf("dining_table" to 0.0),
    )
    return floorPlan.rooms.flatMap { room ->
        val matched = seeds.entries.firstOrNull { (regex, _) -> regex.containsMatchIn(room.label) }
        val centroid = polygonCentroid(room.points)
        matched?.value.orEmpty().map { (catalogId, rot) ->
            FloorPlanLayoutItemDto(catalogId, room.id, listOf(centroid[0], 0.0, centroid[1]), rot)
        }
    }
}
```

STEP 3 Draw each furniture footprint

For each layout item, look up its catalog entry, then draw a rotated rectangle sized from `footprint = [width, depth]`, filled with the catalog's own `color`, optionally with the `icon` emoji drawn centered on top as a quick visual label.

```
fun DrawScope.drawFurniture(
    layout: List<FloorPlanLayoutItemDto>,
    catalogById: Map<String, FurnitureCatalogItemDto>,
    bounds: Bounds
) {
    for (item in layout) {
        val cat = catalogById[item.catalogId] ?: continue
        val (w, d) = cat.footprint
        val center = toOffset(item.position[0], item.position[2], bounds)   // x, z (skip y=height)
        val wPx = (w * PX_PER_METER).toFloat()
        val dPx = (d * PX_PER_METER).toFloat()

        rotate(degrees = item.rotationY.toFloat(), pivot = center) {
            drawRect(
                color = Color(android.graphics.Color.parseColor(cat.color)).copy(alpha = 0.85f),
                topLeft = Offset(center.x - wPx / 2f, center.y - dPx / 2f),
                size = Size(wPx, dPx)
            )
            drawRect(
                color = Color.Black.copy(alpha = 0.25f),
                topLeft = Offset(center.x - wPx / 2f, center.y - dPx / 2f),
                size = Size(wPx, dPx),
                style = Stroke(width = 1f)
            )
        }
        // Optional: draw cat.icon (an emoji string) centered at `center` via drawText/TextMeasurer,
        // the same way room labels were drawn in Section 7 Step 5.
    }
}
```

NOTE

`position` on a layout item is `[x, y, z]` — three values, unlike room points which are `[x, z]` (two values). Index 0 and 2 are the floor-plane coordinates you feed into `toOffset()`; index 1 (height) can be ignored for a top-down 2.5D view.

9. Going Further — True 3D Parity (Optional)

The web app's richest experience — "3D Live" — is an interactive, orbit/fly-camera `@react-three/fiber` (three.js/WebGL) scene where furniture is built from **procedural low-poly meshes** (boxes, cylinders sized from each catalog item's footprint/height/color — no external 3D model files at all), and visitors can even drag furniture around to rearrange it (persisted client-side only, per listing id, in a local layout store).

Matching that fully on Android is a meaningfully larger project than the 2D + overlay MVP in Sections 7-8. If/when it's prioritized:

- **Engine choice:** Google's `Filament` or the higher-level `SceneView` wrapper are the closest native-Android equivalents to three.js for real-time 3D rendering with your own procedural geometry.
- **Reuse everything from Section 7-8:** the exact same `FloorPlanDto`/`FurnitureCatalogItemDto` data models apply — 3D only changes how you turn room polygons into extruded wall meshes and furniture into 3D primitives instead of 2D shapes.
- **Procedural meshes, not asset files:** since the web app never loads external 3D models (`FurnitureMesh.tsx` is a big switch statement building boxes/cylinders per catalog id), you can port that same switch statement to build `Filament` primitives — no art pipeline required.
- **Hotspots** (Section 3.3) become camera-fly-to waypoints — a "jump to room" list driven by each room's hotspot label/order.
- **Scope it as a distinct follow-up phase** rather than blocking the initial floor-plan release on it — the 2D view alone already gives customers the core information (room layout, doors/windows, furniture placement) that drives their decision.

10. Testing Your Implementation

Use the illustrative payload from Section 3.6 as a local fixture (e.g. a JSON asset file loaded in a Compose `@Preview` or an instrumented test) before wiring up a real network call — it's small enough to sanity-check by eye against the checklist below.

Visual checklist (compare against the web app's 2D tab for the same listing, if you have one to reference):

- Both rooms render as separate, correctly-proportioned polygons relative to each other (the bathroom in the sample payload should look noticeably smaller than the bedroom).
- The shared wall between the two rooms is drawn **once**, not twice (this is the #1 bug you'll hit if Section 7 Step 4a's deduplication is skipped or has a bug).
- The door between the rooms shows as a gap in that shared wall with a door-style piece, not a solid uninterrupted wall line.
- The window on the bedroom's outer wall shows as the distinct window styling, not a plain solid line.
- Room labels ("BEDROOM", "BATHROOM") are centered inside their respective rooms, uppercased.
- A listing whose `floorPlan` is JSON null shows your empty state, and does not crash or render a blank rectangle.
- (If doing Section 8) furniture rectangles appear inside their assigned room, are rotated by `rotationY`, and are proportioned according to their real `footprint` — a double bed should visibly dwarf a toilet.

> **NOTE**
>
> Because coordinates are unbounded real-world meters (Section 3.4), also test with a floor plan whose rooms are offset far from the origin (e.g. all coordinates shifted by +50 in x and z) to confirm your bounds calculation (Section 7 Step 1) — not just the origin-anchored sample payload — correctly re-centers the drawing.

Appendix A — Full Field Reference

Consolidated from Section 3 & 4, for quick lookup while coding.

FloorPlanDto

Field	Type	Description
`rooms`	`FloorPlanRoomDto[]`	Every room's outline polygon. See 3.3.
`openings`	`FloorPlanOpeningDto[]`	Every door/window cut into a wall. See 3.3.
`metersPerPixel`	`number (double)`	Audit/provenance value only — the scale used at digitization time to convert the source image into meters. You do NOT need this to render; room/opening coordinates are already in meters.
`sourceFileKey`	`string`	Storage key of the originally uploaded floor-plan image (not needed for rendering).
`sourceFileType`	`string`	MIME type of that source file, e.g. "image/png".
`digitizedAt`	`string (ISO-8601 datetime)`	When the owner/admin traced this floor plan.
`digitizedByRole`	`string`	"owner" or "admin".
`aiProvider`	`string? (nullable)`	Which AI provider assisted digitization, if any (e.g. "gemini"). Informational only.
`hotspots`	`FloorPlanHotspotDto[]? (nullable)`	Named room-to-room navigation waypoints for the 3D scene. Null/empty on plans digitized before 3D support existed — see 8.2 for the fallback.
`allowedCatalogIds`	`string[]? (nullable)`	Which furniture-catalog item ids are appropriate for this floor plan. Null/empty means "no restriction / not set".
`defaultLayout`	`FloorPlanLayoutItemDto[]? (nullable)`	The furniture placement to show by default. Null/empty on older plans — see 8.2 for the auto-seed fallback.

FloorPlanRoomDto

Field	Type	Description
`id`	`string`	Stable room id, referenced by openings' RoomId analogues and by layout items.
`label`	`string`	Human room name, e.g. "Bedroom", "Kitchen".
`points`	`number[][]`	The room's outline as an ordered polygon: each element is a 2-item array `[x, z]`, in **meters**, on the floor plane. Closed implicitly — the last point connects back to the first.

FloorPlanOpeningDto

Field	Type	Description
`id`	`string`	Opening id.
`from`	`number[2]`	Wall-segment start point, [x, z] meters.
`to`	`number[2]`	Wall-segment end point, [x, z] meters.
`position`	`number (0.0–1.0)`	Fraction of the distance from "from" to "to" where this opening is centered — NOT a screen coordinate.
`widthMeters`	`number`	The opening's width, in meters, centered on "position".
`type`	`string`	"door" or "window".

FloorPlanHotspotDto

Field	Type	Description
`id`	`string`	Hotspot id.
`roomId`	`string`	Which room this waypoint belongs to.
`label`	`string`	Display label, e.g. "Bedroom" (used for a "fly to room" list in 3D).
`order`	`int`	Display/navigation order.

FloorPlanLayoutItemDto

Field	Type	Description
`catalogId`	`string`	References an id in the furniture catalog — see Section 4.
`roomId`	`string`	Which room this piece of furniture is placed in.
`position`	`number[3]`	[x, y, z] meters. x/z are the floor-plane position; y is height off the floor (almost always 0 for floor-standing furniture).
`rotationY`	`number (degrees)`	Rotation around the vertical axis, in degrees.

FurnitureCatalogItemDto

Field	Type	Description
`id`	`string`	Stable id, e.g. "bed_double" — this is what defaultLayout.catalogId references.
`name`	`string`	Human display name, e.g. "Double Bed".
`category`	`string`	One of: bedroom, living, dining, kitchen, bathroom.
`footprint`	`number[2]`	**[width, depth]** in meters, on the floor plane (NOT [width, height]).
`height`	`number`	Height in meters — separate from footprint, used for the vertical extent of the 3D mesh.
`color`	`string (hex)`	e.g. "#F5A623" — a fill color for the item's shape (procedural mesh material color, but works equally well as a flat-fill color in a 2D overlay).
`icon`	`string (emoji)`	e.g. a bed emoji character for "bed_double" — handy as a quick visual label if you don't want to draw exact furniture silhouettes.

Appendix B — Full Furniture Catalog

The complete seed catalog as of this writing (15 items), quoted from `src/Application/StaticData/Services/FurnitureCatalogService.cs`. This is what `GET /static/furniture-catalog` returns.

Id	Name	Category	Footprint (w×d, m)	Height (m)	Color	Icon
bed_double	Double Bed	bedroom	1.6 × 2.0	0.55	#F5A623	bed
wardrobe	Wardrobe	bedroom	1.2 × 0.6	1.9	#7C6A58	door
sofa	Sofa (3-seat)	living	2.0 × 0.9	0.75	#FF6B4A	sofa
armchair	Armchair	living	0.8 × 0.8	0.75	#E85A3B	chair
coffee_table	Coffee Table	living	1.0 × 0.5	0.4	#4B5E6B	table
tv_unit	TV Unit	living	1.5 × 0.4	0.5	#1E293B	tv
dining_table	Dining Table	dining	1.6 × 0.9	0.75	#8A6D4E	plate
dining_table_small	Small Dining Table	dining	0.9 × 0.9	0.75	#8A6D4E	plate
dining_chair	Dining Chair	dining	0.45 × 0.45	0.9	#5C4A3A	chair
kitchen_counter	Kitchen Counter	kitchen	2.0 × 0.6	0.9	#94A3B8	basket
fridge	Fridge	kitchen	0.7 × 0.7	1.8	#CBD5E1	ice
stove	Stove	kitchen	0.6 × 0.6	0.9	#334155	fire
toilet	Toilet	bathroom	0.4 × 0.6	0.4	#F8FAFC	toilet
bathroom_sink	Bathroom Sink	bathroom	0.5 × 0.4	0.85	#F8FAFC	faucet
shower	Shower	bathroom	0.9 × 0.9	2.0	#B9D6E0	shower

Icon column above describes the emoji rather than reproducing it, for PDF font-compatibility reasons — the raw API response returns an actual emoji character (e.g. a bed emoji for "bed_double") which renders fine in Kotlin string literals / Android text views.

Appendix C — Quick-Reference Cheat Sheet

Question	Answer
What unit are coordinates in?	Meters. Never pixels, never 0-1 fractions.
Which axes?	x and z (floor plane). A third value "y" (height) only appears on furniture positions, not room/opening points.
Do I need metersPerPixel?	No — it's a provenance/audit field only. Coordinates are already in meters.
What does a listing with no floor plan look like?	"floorPlan": null in the JSON (key present, value null).
Endpoint for a listing	GET /property/listings/{id}
Endpoint for the furniture catalog	GET /static/furniture-catalog (flat array, no pagination)
How do I avoid double-drawing shared walls?	Dedupe edges by a rounded, order-independent endpoint key before drawing (Section 7 Step 4a).
How are doors/windows positioned on a wall?	By opening.position (0-1 fraction along the wall) and opening.widthMeters, not a raw coordinate.
Is furniture placement its own endpoint?	No — it's the defaultLayout field inside a floor plan; the catalog only supplies each item type's definition.
What if defaultLayout is empty?	Older floor plans predate 3D support — auto-seed starter furniture per room by label keyword match (Section 8 Step 2).
Room fill / wall / door / window colors	Room #F8FAFC (no stroke); wall #0F172A; window #3B82F6 (3 lines); door #8A6D4E (line + dashed swing arc).
Web app's px-per-meter constant	40 CSS px/m (PX_PER_METER in floorPlanGeometry.ts) — pick your own dp/m constant for Android.

End of guide. Questions about anything that seems ambiguous or under-specified here should be directed to whoever maintains `StayHereClientApp/src/components/floorplan/` *— that component tree is the living source of truth this document was written from.*